

Document Retrieval Report

Rajat Talwar

November 20, 2025

1 Overview

This assignment implements a vector-space model (VSM) retrieval component for the CACM document collection. It is used to rank documents according to relevance from a set of queries by measuring the similarity between query and document vectors. The system integrates with the provided 'IR_engine.py' file. The logic is implemented in 'my_retriever.py' to achieve a full scoring model supporting *binary*, *tf*, and *tf-idf* term-weighting schemes. Essentially, each query is converted into a dictionary of term counts, which are used to build document vectors and apply the chosen weighting scheme. The system outputs a set of retrieval scores to 'result.txt' and evaluates in 'eval_ir.py' through comparisons with the 'cacm_gold_std.txt' file.

2 Implementation

2.1 Helper Functions and Pre Processing

This section contains descriptions of helper functions and variables created in the initialization of the 'Retrieve' class.

Two arguments are parsed into the creation of a Retrieval object; the inverted index provided as a two-level dictionary and the chosen term weighting scheme.

- 'compute_number_of_documents' - Returns the total number of documents in the collection. Stored in 'self.document_count' to avoid recomputing.
- 'compute_idf' - Returns a dictionary containing the *idf* value for each term in the index using $idf = \log \frac{N}{df}$ where N is the number of documents and df is the number of documents containing the term.
- 'pre_compute_document_vector_lengths' - returns a dictionary with the squared weights of all terms for each document; $\|d\|$. Contains conditional statements that check and include the chosen weighting scheme in the length calculations. This ensures that $\|d\|$ does not have to be calculated at every iteration, reducing runtime.

2.2 Query Processing

The queries are supplied as token lists. These lists are parsed into 'query_dict_of_term_counts' to return a dictionary containing term counts.

2.3 Vector Construction

For each candidate document, a query vector and document vector are constructed over query terms to avoid computing large vectors. This is achieved through the 'compute_i_doc_vector' function where for each term in the query term counts, we search the inverted index for the count of that term in the current document. These counts are appended to a list to create a document vector.

2.4 Applying Weighting Scheme

The constructed document and query vectors are parsed into the 'apply_weighting_scheme' function. Conditional statements check the current weighting scheme and create the weighted document and query vectors accordingly.

2.5 Cosine Similarity

The weighted document and query vector are parsed into the 'cosine_similarity' function and a similarity score is calculated. $\|d\|$ is precomputed as mentioned previously and any zero-vector documents receive a score of 0.

$$\cos(q, d) = \frac{\sum_i q_i d_i}{\|q\| \|d\|}$$

2.6 Ranking

The result for each similarity calculation is added to a dictionary, mapping document id to a cosine value. The dictionary is then sorted and the top 10 documents are returned.

2.7 Performance Analysis

For each weighting scheme, retrieval was evaluated with: no stemming or stop list, stemming, stop list, and both stop list and stemming.

Initially, raw term frequency (*tf*) was used as the weighting scheme for both (*tf*) and (*tf.idf*). While this approach captured term occurrence, it tended to overemphasise very frequent terms, limiting system results. To mitigate this, log normalization of *tf* was applied ($1 + \log(tf)$), producing slightly better F1 scores, as seen in Figure 1 and 2.

-w	—	-p	-s	-s -p
<i>binary</i>	$P : 0.07$	$P : 0.09$	$P : 0.13$	$P : 0.16$
	$R : 0.06$	$R : 0.08$	$R : 0.10$	$R : 0.13$
	$F1 : 0.06$	$F1 : 0.08$	$F1 : 0.11$	$F1 : 0.15$
<i>tf</i>	$P : 0.08$	$P : 0.11$	$P : 0.17$	$P : 0.19$
	$R : 0.06$	$R : 0.09$	$R : 0.13$	$R : 0.15$
	$F1 : 0.07$	$F1 : 0.10$	$F1 : 0.15$	$F1 : 0.17$
<i>tf.idf</i>	$P : 0.21$	$P : 0.26$	$P : 0.22$	$P : 0.27$
	$R : 0.17$	$R : 0.21$	$R : 0.18$	$R : 0.22$
	$F1 : 0.18$	$F1 : 0.23$	$F1 : 0.19$	$F1 : 0.24$
$1 + \log(tf)$	$P : 0.11$	$P : 0.16$	$P : 0.17$	$P : 0.21$
	$R : 0.09$	$R : 0.13$	$R : 0.14$	$R : 0.17$
	$F1 : 0.10$	$F1 : 0.14$	$F1 : 0.16$	$F1 : 0.19$
$1 + \log(tf) * idf$	$P : 0.20$	$P : 0.27$	$P : 0.22$	$P : 0.28$
	$R : 0.16$	$R : 0.21$	$R : 0.17$	$R : 0.22$
	$F1 : 0.18$	$F1 : 0.24$	$F1 : 0.19$	$F1 : 0.25$

Figure 1: Performance Table for Different Weighting Schemes

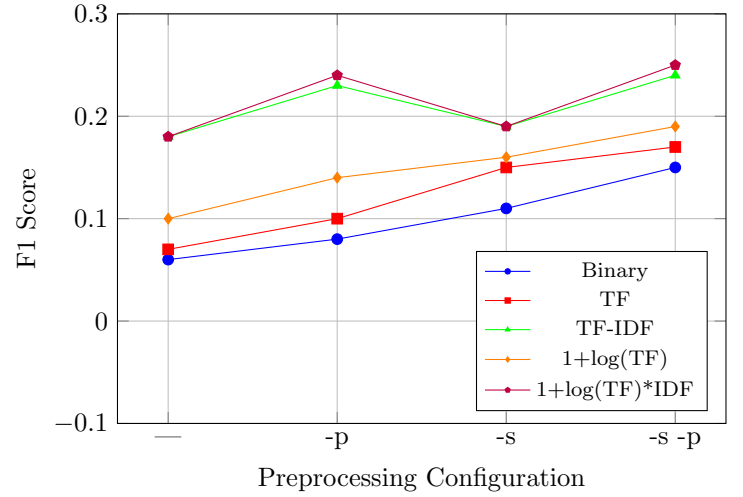


Figure 2: F1 Scores across Preprocessing Configurations

2.7.1 Analysis of Weighting Schemes

A trend can be seen in F1 Scores for weighting schemes that do not use IDF. For non-IDF weighting schemes, the F1 Score is approximately linear from no pre-processing configurations, to both stop list and stemming being applied.

Binary weighting performed the lowest, as it only considers if a term appears or not and ignores term frequency. The F1 score increased by 150% with the addition of stop lists and stemming, demonstrating that the removal of common words and single lemmas improved binary weighting.

Raw tf improved performance by adding term frequency. However, any highly common terms worsened the results. TF saw a 142% increase with the addition of stop listing and stemming, which allowed TF to become more discriminative as many common terms were removed.

A log normalised TF was experimented with to see if better results could be yielded. Figures 1 and 2 show a slight increase between *tf* and $1 + \log(tf)$, with F1 only increasing by 11% with stop lists and stemming applied. This is most likely due to 'document.txt' containing many short documents or very low *tf* values and log normalized *tf* performing better with higher *tf* values.

The introduction of IDF saw major improvements in the system as it took into account how informative a term is across the document collection. Using TF alongside IDF enabled the system to capture term importance in a document and across the collection. As seen in the figures, TF.IDF scored high across all configurations relative to the schemes mentioned previously.

Similar to log normalized *tf*, log normalized *tf.idf* had the best overall performance (Figure 1 and 2). However, $(1 + \log(tf)) * idf$ only increased by 4% compared to *tf.idf* even with stop listing and stemming applied.

It's noteworthy to mention that schemes using IDF saw a slight drop in F1 scores using solely a stop list. It's unclear exactly why this occurred, but it likely could be because the words removed would have helped in achieving a stronger overlap and therefore a higher similarity score.

2.8 Conclusion

Many useful insights can be made from the results gathered from the system such as how different combinations of configurations affect retrieval. However, it's important to highlight the improvement log normalisation had on the results. This could potentially be amplified through scalability; using a larger collection or documents with a larger size. Ultimately, out of the three main weighting schemes, it's clear that *tf.idf* is the best for overall performance and alongside normalization, better results can be yielded.